

Task 1: Day of the Year – Java Program

Aim

To write a Java program to find the **day number of a given date in a year**.

Algorithm

1. Start the program.
2. Read the **date, month, and year** from the user.
3. Store the number of days in each month.
4. Check whether the given year is a **leap year**.
5. If it is a leap year, set February to 29 days.
6. Add the number of days in all months before the given month.
7. Add the given date to the total.
8. Display the calculated day of the year.
9. Stop the program.

Program

```
import java.util.Scanner;

public class DayOfYear {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter day: ");

        int day = sc.nextInt();

        System.out.print("Enter month: ");

        int month = sc.nextInt();

        System.out.print("Enter year: ");

        int year = sc.nextInt();
```

```

int[] daysInMonth = {
    31, 28, 31, 30, 31, 30,
    31, 31, 30, 31, 30, 31
};

// Check for leap year
if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
    daysInMonth[1] = 29;
}

int dayOfYear = 0;

for (int i = 0; i < month - 1; i++) {
    dayOfYear += daysInMonth[i];
}

dayOfYear += day;

System.out.println("Day of the Year: " + dayOfYear);

sc.close();
}
}

```

Output

Example 1:

Enter day: 18

Enter month: 8

Enter year: 2026

Day of the Year: 230

Example 2 – Leap Year:

Enter day: 29

Enter month: 2

Enter year: 2024

Day of the Year: 60

Result

Thus, the Java program to **find the day of the year for a given date** was successfully executed and the correct day number was obtained.

Task 2: Day of the Week – Java Program

Aim

To write a Java program to find the **day of the week** for a given date.

Algorithm

1. Start the program.
2. Read the **day, month, and year** from the user.
3. Use Java's LocalDate class to create the given date.
4. Use the getDayOfWeek() method to find the day of the week.
5. Display the day of the week.
6. Stop the program.

Program

```
import java.util.Scanner;
import java.time.LocalDate;

public class DayOfWeek {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter day: ");
        int day = sc.nextInt();

        System.out.print("Enter month: ");
        int month = sc.nextInt();

        System.out.print("Enter year: ");
        int year = sc.nextInt();

        LocalDate date = LocalDate.of(year, month, day);
```

```
System.out.println("Day of the Week: " + date.getDayOfWeek());
```

```
sc.close();
```

```
}
```

```
}
```

Output

Enter day: 18

Enter month: 8

Enter year: 2026

Day of the Week: TUESDAY

Result

Thus, the Java program to **find the day of the week for a given date** was successfully executed and the correct day of the week was obtained.

Task 3: Largest Number – Java Program

Aim

To write a Java program to find the **largest among three numbers**.

Algorithm

1. Start the program.
2. Read three numbers from the user.
3. Compare the first number with the other two numbers.
4. If the first number is greater than both, it is the largest.
5. Otherwise, compare the second number with the third number.
6. Display the largest number.
7. Stop the program.

Program

```
import java.util.Scanner;

public class LargestNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.print("Enter third number: ");
        int c = sc.nextInt();

        int largest;
```

```
    if (a >= b && a >= c) {  
        largest = a;  
    } else if (b >= a && b >= c) {  
        largest = b;  
    } else {  
        largest = c;  
    }  
  
    System.out.println("Largest Number: " + largest);  
  
    sc.close();  
}  
}
```

Output

Enter first number: 25

Enter second number: 78

Enter third number: 42

Largest Number: 78

Result

Thus, the Java program to **find the largest among three numbers** was successfully executed and the largest number was obtained.

Task 4: Sort the People – Java Program

Aim

To write a Java program to **sort the names of people in alphabetical order**.

Algorithm

1. Start the program.
2. Read the number of people.
3. Read the names of all people.
4. Store the names in an array.
5. Compare the names using alphabetical order.
6. Sort the names using Arrays.sort().
7. Display the sorted names.
8. Stop the program.

Program

```
import java.util.Arrays;
import java.util.Scanner;

public class SortPeople {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of people: ");
        int n = sc.nextInt();
        sc.nextLine();

        String[] people = new String[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter name " + (i + 1) + ": ");
```

```
        people[i] = sc.nextLine();
    }

    Arrays.sort(people);

    System.out.println("\nPeople in alphabetical order:");

    for (String name : people) {
        System.out.println(name);
    }

    sc.close();
}
```

Output

Enter number of people: 5

Enter name 1: Rahul

Enter name 2: Arun

Enter name 3: Karthik

Enter name 4: Bala

Enter name 5: Deepak

People in alphabetical order:

Arun

Bala

Deepak

Karthik

Rahul

Result

Thus, the Java program to **sort the names of people in alphabetical order** was successfully executed and the names were displayed in sorted order.

